

3-Month Backend SDE-2 Preparation Syllabus

Target Companies: Razorpay, J.P. Morgan, Morgan Stanley, Oracle, Tier-1 Product Firms

TARGET ROLE	EXPERIENCE BRACKET	CORE STACK	TIMELINE
SDE-2 / Senior Backend	4.5 Years Experience	Java, Spring Boot, Postgres	12 Weeks (3 Months)

Strategic Positioning: With 4.5 YOE, top product companies evaluate on core computer science foundations, low-level design, database query execution mechanics, and distributed resilience. Focus 90% of preparation on Backend, Concurrency, and System Design; maintain 10% on React for architectural full-stack discussions.

TOPIC 01 CORE JAVA & JVM INTERNALS (Sequence of Learning)

STEP	SUBTOPIC	WHAT YOU NEED TO LEARN IN DEPTH	PRIORITY
01	Java 8–21 Core Features	Functional interfaces, Stream API (lazy evaluation, flatMap, groupingBy), CompletableFuture, Records, Pattern Matching, Sealed Classes.	High
02	Collections Internals	HashMap & ConcurrentHashMap (hash bucket mechanics, treeification, CAS + bin locks), CopyOnWriteArrayList, TreeMap (Red-Black tree), equals() & hashCode() contracts.	Critical
03	Multithreading & Concurrency	Java Memory Model (JMM), volatile, synchronized monitor locks, CAS & Atomic*, ReentrantLock, ReadWriteLock, Deadlock detection.	Critical
04	java.util.concurrent (JUC)	ThreadPoolExecutor (core/max pool tuning, work queues, rejection policies), CountdownLatch, CyclicBarrier, Semaphore, BlockingQueue, Virtual Threads (Project Loom).	Critical
05	JVM Memory & GC Mechanics	Heap layout (Young: Eden/Survivor, Old Gen), Metaspace, Stack. GC algorithms (G1 GC, ZGC, generational pauses, STW). Memory leaks and thread/heap dump analysis.	High

TOPIC 02 SPRING BOOT & MICROSERVICES INTERNALS (Sequence of Learning)

STEP	SUBTOPIC	WHAT YOU NEED TO LEARN IN DEPTH	PRIORITY
01	IoC, DI & Bean Lifecycle	ApplicationContext bootstrapping, Bean creation lifecycle callbacks (@PostConstruct, BeanPostProcessor), Singleton vs Prototype thread safety, Circular dependency resolution.	High
02	Spring AOP & Proxies	JDK Dynamic Proxy vs CGLIB, how @Transactional and @Async work under the hood, why self-invocation bypasses transactional proxies and how to fix it.	Critical
03	Auto-Configuration Internals	@SpringBootApplication decomposition, @ConditionalOnProperty / @ConditionalOnClass, writing and debugging custom starter modules.	Medium
04	Microservices Architecture	Service discovery, API Gateway routing & filter chains, Synchronous (Feign/REST/gRPC) vs Asynchronous messaging, Spring Cloud / containerized microservices.	High
05	Resilience & Observability	Circuit Breakers (Resilience4j), Retry with exponential backoff, Bulkhead isolation, Rate limiting, Distributed tracing (Micrometer, OpenTelemetry, Zipkin), Spring Actuator.	Critical

TOPIC 03 POSTGRESQL & SPRING DATA JPA (Sequence of Learning)

STEP	SUBTOPIC	WHAT YOU NEED TO LEARN IN DEPTH	PRIORITY
01	Hibernate & JPA Mechanics	Entity lifecycle states (Transient, Persistent, Detached, Removed), First-level vs Second-level cache, Dirty checking mechanism.	High
02	The N+1 Query Problem	Root causes in Lazy vs Eager loading; fixing via JOIN FETCH, @EntityGraph, batch size tuning (hibernate.default_batch_fetch_size).	Critical
03	Transactions & Isolation	ACID properties, Isolation levels (Read Committed, Repeatable Read, Serializable), Read phenomena (Dirty, Non-repeatable, Phantom reads), Propagation levels (REQUIRED, REQUIRES_NEW).	Critical
04	Locking & Concurrency Control	Optimistic Locking with @Version vs Pessimistic Locking (PESSIMISTIC_WRITE / SELECT FOR UPDATE), preventing double-spending and seat-overbooking.	Critical
05	Postgres Indexing & Tuning	B-Tree, Hash, GIN indexes; Composite index column order; Covered queries; Reading and interpreting EXPLAIN ANALYZE; Table partitioning.	High

TOPIC 04 LOW-LEVEL DESIGN & MACHINE CODING (Sequence of Learning)

STEP	SUBTOPIC	WHAT YOU NEED TO LEARN IN DEPTH	PRIORITY
01	Object-Oriented & SOLID Principles	Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion; DRY, KISS, Loose Coupling, High Cohesion.	Critical
02	GoF Design Patterns	Strategy, Factory / Abstract Factory, Observer, Decorator, Builder, State, Singleton (Thread-safe double-checked locking), Chain of Responsibility.	Critical
03	Machine Coding Boilerplate & Schema	Project packaging: controller → service → repository → model → strategy. In-memory data store with thread safety, writing clean JUnit tests.	High
04	Standard LLD Problem Implementations	Implement from scratch in 90 mins: (1) Rate Limiter (Token Bucket), (2) Parking Lot, (3) Splitwise, (4) Movie Ticket Booking (BookMyShow with seat locks), (5) Payment Gateway Router.	Critical

TOPIC 05 HIGH-LEVEL DESIGN & DISTRIBUTED SYSTEMS (Sequence of Learning)

STEP	SUBTOPIC	WHAT YOU NEED TO LEARN IN DEPTH	PRIORITY
01	Architectural Building Blocks	Load Balancers (L4 vs L7, round-robin, least connections), Reverse Proxies, CDNs, Consistent Hashing (Virtual Nodes), CAP Theorem, PACELC.	High
02	Distributed Caching (Redis)	Cache-Aside, Write-Through, Write-Back; Cache stampede / avalanche prevention; Eviction policies (LRU, LFU, TTL); Distributed locks via Redlock / SETNX.	Critical
03	Event Streaming & Messaging (Kafka)	Topics, Partitions, Consumer Groups, Offset commits, Exactly-once vs At-least-once semantics, Dead Letter Queues (DLQ), handling message reordering.	Critical
04	FinTech & Distributed Consistency	Idempotency design (Unique idempotency keys + deduplication store), Saga Pattern (Orchestration vs Choreography), Transactional Outbox Pattern, Double-entry ledgers.	Critical
05	Standard HLD Case Studies	Design: (1) URL Shortener (TinyURL), (2) Distributed Notification System, (3) Payment Processing & Webhook Engine (Razorpay style), (4) Flash Sale / Inventory Lock.	High

TOPIC 06 DATA STRUCTURES & ALGORITHMS (Sequence of Learning)

STEP	SUBTOPIC	CORE PATTERNS & LEETCODE MEDIUM FOCUS	PRIORITY
01	Arrays & Two Pointers	Two Pointers, Sliding Window (fixed/variable), Prefix Sum, 3Sum, Container With Most Water, Minimum Size Subarray Sum.	High
02	Hashing & Binary Search	HashMap frequency counters, Group Anagrams, Subarray Sum Equals K; Binary search on sorted arrays, Rotated sorted array search, Search on answer space.	High
03	Linked Lists & Stacks/Queues	Fast & Slow pointers (Cycle detection), Reverse linked list, Monotonic Stack (Next Greater Element, Daily Temperatures, Largest Rectangle).	Medium
04	Trees & Binary Search Trees	DFS (Pre/In/Post order), BFS (Level order, Zigzag), Lowest Common Ancestor (LCA), Diameter, Validate BST, Serialize/Deserialize Binary Tree.	Critical
05	Graphs & Heaps	BFS/DFS on Grid (Number of Islands), Cycle Detection, Topological Sort (Course Schedule), Dijkstra; Top K Frequent Elements, Merge K Sorted Lists.	Critical
06	Dynamic Programming (Core)	0/1 Knapsack variations, Coin Change (min coins, total combinations), Longest Increasing Subsequence (LIS), House Robber, Word Break.	High

TOPIC 07 REACT & FULL-STACK INTEGRATION (Sequence of Learning)

STEP	SUBTOPIC	WHAT YOU NEED TO LEARN IN DEPTH	PRIORITY
01	Core Mechanics & Lifecycle	Virtual DOM reconciliation, Component render lifecycles, JSX compilation, Functional vs Class components.	Medium
02	Essential React Hooks	useState, useEffect (dependency array behavior & cleanups), useMemo, useCallback for performance optimization, useRef.	High
03	State Management & Data Flow	Props & Prop Drilling, Context API vs Redux Toolkit / Zustand, Global client state vs Server state caching (React Query / TanStack).	Medium
04	Full-Stack Security & Network	Axios interceptors, JWT storage strategy (HttpOnly Cookies vs LocalStorage XSS risks), CSRF token handling, CORS policy resolution on Spring Gateway.	High

ROADMAP 12-WEEK EXECUTION SCHEDULE

Phase & Weeks	Primary Technical Focus	Milestone / Deliverable
Month 1 (Weeks 1–4)	- Core Java Concurrency, JUC, Memory Model, JVM internals - SOLID Principles & Design Patterns - DSA: Arrays, Strings, Two Pointers, Binary Search, Linked Lists	- Solve 40 LeetCode Mediums - Implement 2 LLD problems in IDE from scratch (Rate Limiter, Parking Lot)
Month 2 (Weeks 5–8)	- Spring Boot Internals (AOP, Bean Lifecycle, Transactions) - Postgres Query Tuning, Indexing, N+1 problem, Locking - Redis Caching & Kafka Event Streaming - DSA: Trees, BST, Heaps, Graphs (BFS/DFS)	- Solve 40 LeetCode Mediums (Trees/ Graphs) - Implement 2 LLD problems (BookMyShow, Splitwise) - Draft HLD designs for Notification & TinyURL
Month 3 (Weeks 9–12)	- High-Level Design: Payment Gateway, Idempotency, Saga - React Full-Stack Integration & Security review - DSA: Dynamic Programming core patterns & Revision - STAR Resume refactoring & Aggressive Referral Applications	- Conduct 3 full mock interviews (DSA, LLD, HLD) - Apply via LinkedIn employee referrals for SDE-2 roles